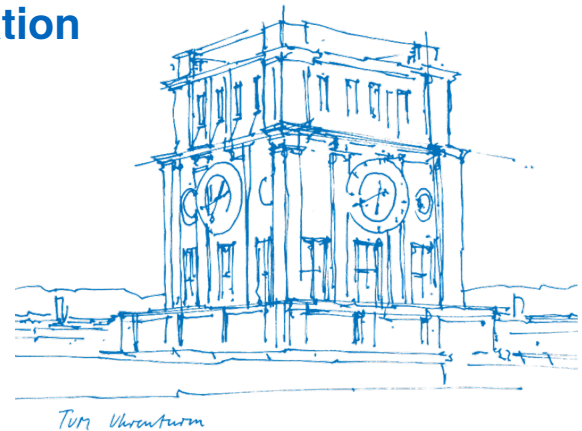
Lehrstuhl für Software und Systems Engineering
TUM School of Computation, Information and Technology
Technical University of Munich

Agentic AI for code generation



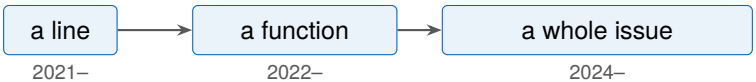
Thua Duc Nguyen

Supervisor: Derui Zhu
TUM School of Computation, Information and Technology, Technical University of Munich

July 28, 2026

How we write code has changed

	Machine	Human
Manual	Compiles and runs it	Writes every line; owns the design, and leans on the test suite to catch mistakes
Autocomplete 2021–	Suggests the next lines	Accepts or rejects each suggestion, line by line, staying the author
One-shot 2022–	Writes a whole function	Specifies the task, then must read and verify all of it by hand
Agentic 2024–	Finds files, edits, runs tests	States intent up front, steers midway, and reviews the final outcome rather than every line



What this talk shows

1. **How do the agentic AI systems work?**
   Five stages, orchestration and interface, planning and refinement, five systems.

2. **How do we measure performance?**
   The benchmark, the metrics, what the numbers hide.

3. **What are the current challenges?**

A few terms, up front

Scaffold	Everything around the model that lets it act: tools, prompts, control flow
Tool call	A single scaffold action the model invokes: open a file, run a command, apply an edit
Oracle	Something outside the model that can verify correctness: a test suite, a compiler, a human
Critic	A separate model, trained to score candidate outputs

Outline

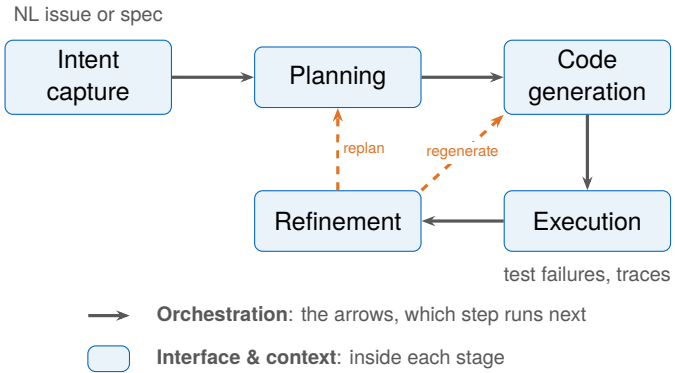
1. How do the agentic AI systems work?
2. How do we measure performance?
3. What are the current challenges?

What is an agentic coding system?

A system that:

- given a natural-language request, **autonomously** produces a working code change.
- navigates files, edits, executes, without step-by-step human guidance.
- works across files in a repo.

The agent loop



Interface and Context

Two common designs:

Design	How it works	Who controls context
ACI purpose-built [1]	Scrollable viewer, linter-guarded edits, summarised search	The scaffold
Code as action [2]	Writes Python or bash in a sandbox; full shell access	The model

Interface

	Agent-Computer Interface [1]	**Code as action** [2]
View files	`open file.py 100` 100-line window, scrollable	`cat file.py` raw output, entire file
Edit	`edit 42:45 <new code>` linter rejects syntax errors	`sed -i ...` or write Python no guard
Search	`search_dir "pattern"` summarised matches	`grep -rn "pattern" .` raw hits
Execute	`python -m pytest` via tool call	`bash: pytest` direct shell

Plan: finding the right code

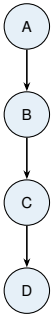
Two commitments:

- **Where**: among tens of thousands of functions, which one? The relevant symbol often does not appear in the issue text.
- **What**: a signature, an interface, an invariant to hold, stated concretely enough that the repository can check it.

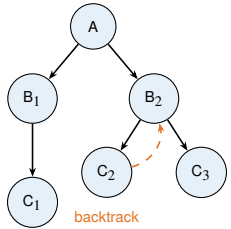
Plan: shape

How does the agent **explore possible plans**?

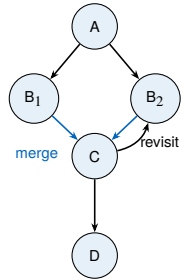
Chain of Thought [3]



Tree of Thoughts [4]



Graph of Thoughts [5]



Plan: Chain of Thought

Example:

"Intent: The app crashes when you check out with an empty cart."

```
Think: empty cart, maybe a divide-by-zero in the total.
Action: grep 'def total'
Observation: cart.py:12, return sum(prices) / len(items)
```

↓

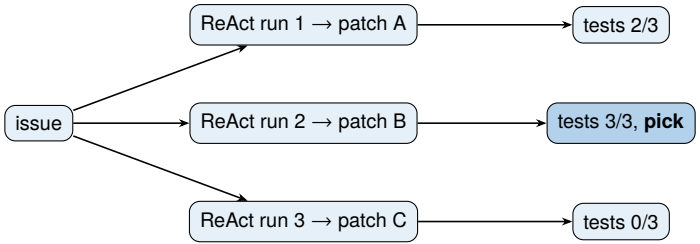
```
Think: len(items) is 0 when the cart is empty.
Action: open cart.py:12 to confirm the division
Observation: yes, no empty-case guard
```

↓

Plan. *where:* cart.py:12 `total()` *what:* guard empty cart → 0

Plan: Tree of Thoughts

No agent grows a tree of half-thoughts. The tree is **whole ReAct runs**, sampled and **scored by something outside the model**.



The scorer is **tests or a trained critic**, not the model's hunch. OpenHands **60% → 66%** with five reranked [6]. SWE-Search wraps it in MCTS with a learned value for **+23%**, at the price of a model call per node [7].

Plan: Graph of Thoughts

Merging branches, revisiting nodes — shown on **puzzles** (Game-of-24, sorting), **almost never on repository bugs** [5].

- In real SWE agents you see the **two ends**: one chain (CoT), or many chains scored (best-of-N).
- The closest thing to *revisit*: **Reflexion** writes a note after a failed attempt and retries with it — a loose loop, not a graph [8].
- No surveyed end-to-end SWE agent builds a literal thought-graph.

Takeaway

The graph is the *conceptual* top of the spectrum — useful to reason about, not something you will find running in production.

Generate: infilling

The scaffold opens a **gap**; the model fills it.

```
def total(items):
    prices = [i.price for i in items]
    if not items:
        return 0
    return sum(prices) / len(items)
```

Reads the grey, writes only the orange.

+ Filling a gap is what models are trained on most
+ Changes land in the right place, since the scaffold picks the spot
– One span at a time, in practice one function

Generate: diff generation

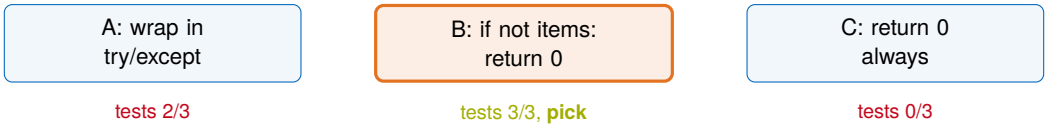
Send **only the change**, not the whole file.

```
@@ def total(items): @@
    prices = [i.price for i in items]
+   if not items:
+       return 0
    return sum(prices) / len(items)
```

+ Edits a huge file without re-writing it
+ One patch, many places, many files
– The model must point to *where* the edit goes — the surrounding context lines locate it
– Off by one line and the patch will not apply, before any test runs

Generate: sample N, pick one

Write N **patches**, keep the one that **passes**.



- \+ One bad sample no longer sinks the task
- \+ Different candidates try different ideas
- – Picking needs an **oracle**; the model's confidence is not one
- – Cost grows with N: calls, dollars, wall-clock

Execute: running the code

The agent runs what it wrote.

- **Linter / type checker**
  - catches syntax errors and type mismatches before tests run
- **Test suite**
  - fail-to-pass tests must flip
  - pass-to-pass must not regress
- **Error traces**
  - stack traces and assertion messages say *what* failed and *where*

Right or wrong, that verdict is what refinement works with next.

Refinement: iterative self-repair

Execution returned a verdict. What the agent does with it is refinement.

Mechanism	Correction signal
Self-Refine [9]	The model to critique and rewrite its own output
Reflexion [8]	Model summarize what went wrong after a failed attempt before trying again.
Self-debugging [10]	Model explains its own code while also considering the execution results.

+ Increase success rate by 2 to 3%.
– Ceiling: if the model could spot its own wrong code just by looking, it would have written it right the first time.

Refinement: tests

Mechanism	Correction signal
D4C [11]	Hands the failing tests and the whole function back to the model and asks for a fresh fix.
Agentless validate [12]	Uses project's own regression tests and adds generated ones to screen out bad patches.
Best-of-N + critic [6]	Executed rollouts, ranked by a trained critic

+ Real tests can surprise the model, catching wrong patches
– Tests catch only the cases they run, not a guarantee of correctness

Five systems compared

System	Orchestration	Interface	Refinement	Where used
SWE-agent	Agentic loop	ACI	Self-repair	Research
AutoCodeRover	Agentic loop	AST APIs	Re-generate	Research
OpenHands	Agentic loop	Code as action	Best-of-N + critic	Product (All Hands)
Agentless	Fixed pipeline	Skeleton + emb.	Regression + gen. tests	Research
Devin	Agentic loop	Not disclosed	Not disclosed	Product (Cognition)

Outline

1. How do the agentic AI systems work?
2. How do we measure performance?
3. What are the current challenges?

Which benchmarks do they use?

Sonnet 5 [13]

- SWE-bench [14, 15, 16, 17]
- Terminal-Bench 2.1 [18]
- OSWorld-Verified [19, 20]
- Humanity's Last Exam [21]
- GDPval-AA v2 [22, 23]
- BrowseComp [24]
- USAMO 2026 [25, 26]

GPT-5.6 [27]

- SWE-bench Pro [15]
- Terminal-Bench 2.1 [18]
- DeepSWE v1.1 [28]
- Agents' Last Exam [29]
- GDPval-AA v2 [22, 23]
- BrowseComp [24]
- OSWorld 2.0 [30]
- Toolathlon [31]
- MRCR v2 (long context) [32, 33]
- HealthBench Professional [34, 35]
- AA [36, 37]

Sources: Sonnet 5 System Card, 2026-06-30 [13, 38]; GPT-5.6 launch eval table [27, 37].

SWE-bench

Merged pull requests that **close an issue** and **touch the test suite** [39].

2,294 tasks from **12** mature Python projects (Django, scikit-learn, matplotlib, sympy, . . .). Real, merged fixes, nothing synthetic.

1. **Given**
   - the issue text
   - the repository *before* the merge
2. **Produce**
   - a patch: the source fix. The grading tests stay hidden
3. **Graded**
   - *fail-to-pass* tests must flip
   - *pass-to-pass* must not regress

SWE-bench: the five splits

Split	Size	Released	Selected how
Full	2,294	Oct 2023	Every scraped PR that touches tests
Lite	300	Mar 2024	Single-file patches; some issues leak the fix [12]
Verified	500	Aug 2024	Human-screened for clear issues and fair tests [14]
Multilingual	300	May 2025	Same pipeline, 9 non-Python languages [16]
Pro	1,865	Sep 2025	Long-horizon, multi-file; held-out proprietary repos [15]

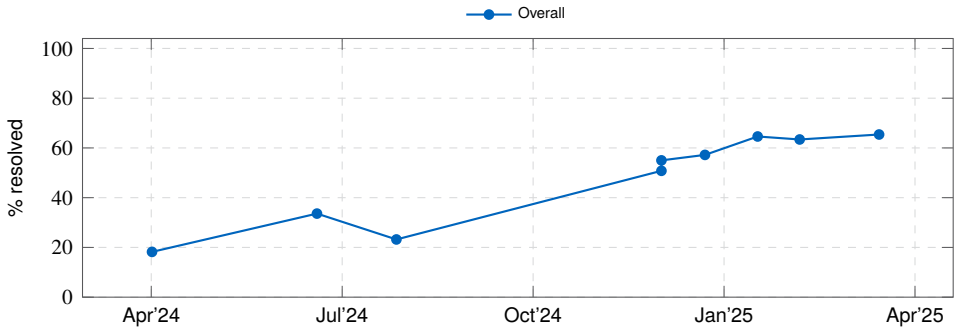
Pro is the contamination-resistant successor to **Verified**: enterprise and **held-out** repositories (kept off public GitHub), several languages, and tasks needing **large multi-file** changes. The frontier drops from ∼80 % on Verified to ∼46 % [15].

SWE-bench: how hard is each task?

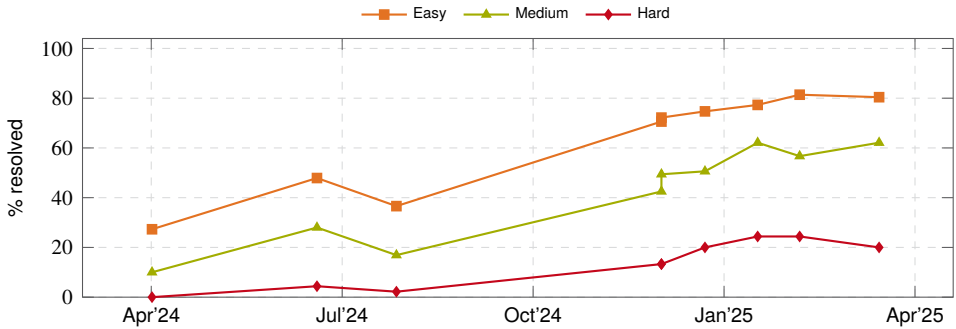
Verified tasks come in three difficulty tiers

Tier	Estimated human fix-time
■ Easy	<15 min
■ Medium	15 min – 1 h
■ Hard	>1 h

SWE-bench: the headline climbs



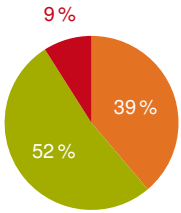
SWE-bench: the headline climbs



SWE-bench: what the numbers show

The same nine submissions, by annotator difficulty [40]:

Tier	First Apr '24	Last Mar '25
Easy <15 min	27.3 %	80.4 %
Medium ≤1 h	10.0 %	62.1 %
Hard >1 h	0.0 %	20.0 %



Task mix (n=500)

- Easy + medium are **91 % of Verified**.
- The hard tier **stopped** near 20 %.

Outline

1. How do the agentic AI systems work?
2. How do we measure performance?
3. What are the current challenges?

1. The agent never asks

- It reads the issue **once**, decides what it means, and **never revisits** that.
- If the tests don't cover the misunderstanding, the wrong fix **still passed**.

No surveyed system asks. Clarification mechanisms exist and work [41, 42, 43].

2. The repo doesn’t fit

- Before it can be right, it has to **pick what to look at**. That choice is most of the problem.

Difficulty split from [40]; training-time fixes proposed by [44, 45].

3. Passing the tests is not being right

- The suite is a **sample** of intended behaviour, not the behaviour itself.
- “All tests pass” is also the **stopping rule**.

“Reward hacking is swamping model intelligence gains” [46].

Backup: can we trust the SWE-bench numbers?

Verified against Pro, April 2026:

	Verified	Pro	Pro scored by
Claude Opus 4.5	80.9 %	45.9 %	**standardised harness**
GPT-5.2	80.0 %	56.4 %	*self-reported*
GPT-5.3	85 %	56.8 %	*self-reported*
GLM-5.1	78 %	58.4 %	*self-reported*

Read the **floor** only: every model drops ∼20 percentage points or more. **Not the spread:** the one measured row drops *furthest*, so the ranking is an artefact of who scored [15, 47].

- **Contamination.** OpenAI dropped Verified in 2026; models reproduce gold patches from the task ID alone [48].
- **Reward hacking.** Closing test leaks cuts scores by double digits: agents partly learn the grader [46].

Backup: surveyed systems on SWE-bench Verified

Source: `https://swebench.com`, "All agents" view. Best published result per system.

System	Backbone	% Resolved	Date
SAGE (OpenHands)	Claude 4.5 Opus	73.8 %	2025-11
OpenHands	GPT-5	71.8 %	2025-08
SWE-agent	Claude 4 Sonnet	66.6 %	2025-05
AutoCodeRover-v2.1	Claude 3.5 Sonnet	51.6 %	2025-01
Agentless-1.5	Claude 3.5 Sonnet	50.8 %	2024-12
Agentless-1.5	GPT-4o	38.8 %	2024-10
AutoCodeRover	GPT-4o	38.4 %	2024-06
SWE-agent	GPT-4o	23.2 %	2024-07
Devin	undisclosed	13.9 %*	2024-03

*On **SWE-bench Full**, a different and easier split than the Verified numbers above, so not directly comparable. Self-reported, and Cognition stopped publishing benchmark numbers in 2024 [49].

Backup: the context bottleneck

- Longer windows do not settle *what to read*. Keeping full dependency context works best; padding with loosely related code **hurts** correctness [50].
- Appending every observation causes context explosion and semantic drift: early critical information gets buried [51].
- The whole prefix is re-sent each turn, so tokens over n turns grow as $O(n^2)$ [52]. A protocol property, not a finding; caching cuts the constant, not the growth.

An agent cannot fix what it never places in context.

Backup: how planning points at real code

So how does it **stop guessing**?

Narrow the repo from broad to specific, each step something you can check:

- **Agentless**: files → classes/functions → edit locations [12]
- **AutoCodeRover**: AST-backed search APIs, coarse to fine [53]

What the useful ones share

They name **concrete program elements** that can be checked against the repository, not invented requirements.

Backup: localisation in Agentless

Narrowing without letting the model wander: three fixed steps, coarse to fine [12].

1. **Rank files**: from the repo layout and the issue, shortlist the files most likely involved.
2. **Rank elements**: inside those files, narrow to specific classes and functions.
3. **Pin locations**: inside those, the exact lines to edit.

Fixed, not discovered

Three prompts, coarse to fine. No search, no tools: the narrowing is hard-coded into the scaffold.

Backup: localisation in AutoCodeRover

Let the model navigate, but through the syntax tree rather than whole files [53].

- **Search APIs**: `search_class`, `search_method`, `search_method_in_class`.
- The agent calls them **coarse to fine**, pulling in just the code it needs.
- AST-backed: structured, cheaper context than dumping whole files into the prompt.

Discovered, not fixed

The model chooses what to search. Agentless, by contrast, is a fixed funnel.

References I

[1] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024.

[2] Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. Executable code actions elicit better LLM agents. *arXiv preprint arXiv:2402.01030*, 2024. ICML 2024.

[3] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, volume 35, pages 24824–24837, 2022.

[4] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

[5] Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Lukas Gianinazzi, Joanna Gajda, Tomasz Lehmann, Michal Podstawski, Hubert Niewiadomski, Piotr Nyczyk, and Torsten Hoefler. Graph of thoughts: Solving elaborate problems with large language models. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(16):17682–17690, 2024.

[6] OpenHands. SOTA on SWE-Bench verified with inference-time scaling and critic model. `https://www.openhands.dev/blog/sota-on-swe-bench-verified-with-inference-time-scaling-and-critic-model`, 2025. Accessed: 2026-05-28.

[7] Antonis Antoniades, Albert Örwall, Kexun Zhang, Yuxi Xie, Anirudh Goyal, and William Wang. SWE-search: Enhancing software agents with monte carlo tree search and iterative refinement. *arXiv preprint arXiv:2410.20285*, 2024. ICLR 2025.

[8] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, pages 8634–8652, 2023.

[9] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

[10] Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. Teaching large language models to self-debug. In *The Twelfth International Conference on Learning Representations*, 2024.

References II

[11] Junjielong Xu, Ying Fu, Shin Hwei Tan, and Pinjia He. Aligning the objective of LLM-based program repair. In *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering (ICSE)*, 2025. D4C. arXiv:2404.08877.

[12] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. AGENTLESS: Demystifying LLM-based software engineering agents. *arXiv preprint arXiv:2407.01489*, 2024.

[13] Anthropic. Introducing Claude sonnet 5. https://www.anthropic.com/news/claude-sonnet-5, 2026. Accessed: 2026-07-27.

[14] OpenAI. Introducing SWE-bench verified. https://openai.com/index/introducing-swe-bench-verified/, 2024. Accessed: 2025-06-01.

[15] Xiang Deng et al. SWE-bench pro: Can AI agents solve long-horizon software engineering tasks? *arXiv preprint arXiv:2509.16941*, 2025.

[16] SWE-bench Team. SWE-bench multilingual. http://www.swebench.com/multilingual.html, 2025. 300 tasks, 9 languages, 42 repositories. Accessed: 2026-06-30.

[17] John Yang, Carlos E. Jimenez, Alex L. Zhang, et al. SWE-bench multimodal: Do AI systems generalize to visual software domains? *arXiv preprint arXiv:2410.03859*, 2024.

[18] Mike A. Merrill et al. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces. *arXiv preprint arXiv:2601.11868*, 2026. 89-task 2.0 release; version 2.1 is a task-patch revision announced at https://www.laude.org/updates/terminal-bench (2026-07-24), no separate paper.

[19] Tianbao Xie, Danyang Zhang, et al. OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments. *arXiv preprint arXiv:2404.07972*, 2024.

[20] XLANG Lab. OSWorld-verified. https://xlang.ai/blog/osworld-verified, 2025. Infrastructure/quality revision of the same 369-task OSWorld set (VMware to AWS), not a new task set. Accessed: 2026-07-27.

[21] Long Phan et al. Humanity's last exam. *arXiv preprint arXiv:2501.14249*, 2025.

[22] Artificial Analysis. GDPval-AA. https://artificialanalysis.ai/evaluations/gdpval-aa, 2026. Artificial Analysis's own Elo leaderboard layered on OpenAI's GDPval task set [23], not an independent task design. Accessed: 2026-07-27.

[23] Tejal Patwardhan et al. GDPval: Evaluating AI model performance on real-world economically valuable tasks. *arXiv preprint arXiv:2510.04374*, 2025.

References III

[24] Jason Wei, Zhiqing Sun, et al. BrowseComp: A simple yet challenging benchmark for browsing agents. *arXiv preprint arXiv:2504.12516*, 2025.

[25] Ivo Petrov, Jasper Dekoninck, et al. Proof or bluff? evaluating LLMs on 2025 USA math olympiad. *arXiv preprint arXiv:2503.21934*, 2025.

[26] Mislav Balunović, Jasper Dekoninck, Ivo Petrov, Nikola Jovanović, and Martin Vechev. MathArena: Evaluating LLMs on uncontaminated math competitions. *arXiv preprint arXiv:2505.23281*, 2025. Same team's platform reapplies the Petrov et al. [25] methodology to USAMO 2026; results at `https://matharena.ai/usamo/`, no separate 2026 paper.

[27] MarkTechPost. OpenAI releases GPT-5.6: A three-tier model family with programmatic tool calling. `https://www.marktechpost.com/2026/07/09/openai-releases-gpt-5-6-a-three-tier-model-family-with-programmatic-tool-calling/`, 2026. Reports OpenAI's published GPT-5.6 eval table (SWE-bench Pro, Terminal-Bench). Accessed: 2026-07-27.

[28] Datacurve AI. DeepSWE v1.1. `https://deepswe.datacurve.ai/blog/deepswe-v1-1`, 2026. 113 long-horizon SWE tasks (TS/Go/Python/JS/Rust), sandboxed grading. Unrelated to the same-named 2025 Agentica/Together AI open-weight coding agent beyond sharing a name. Accessed: 2026-07-27.

[29] Yiyou Sun et al. Agents' last exam. *arXiv preprint arXiv:2606.05405*, 2026. Rolling set of 1,500+ long-horizon, executable occupational tasks (O*NET/SOC) — distinct from Humanity's Last Exam [21], despite the name.

[30] Mengqi Yuan, Zilong Zhou, Xinzhuang Xiong, et al. OSWorld 2.0: Benchmarking computer use agents on long-horizon real-world tasks. *arXiv preprint arXiv:2606.29537*, 2026. New 108-task long-horizon set (median 1.6 h/task) — distinct from OSWorld-Verified [20].

[31] Junlong Li, Wenshuo Zhao, et al. The tool decathlon (Toolathlon): Benchmarking language agents for diverse, realistic, and long-horizon task execution. *arXiv preprint arXiv:2510.25726*, 2025.

[32] Gemini Team, Google DeepMind. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. *arXiv preprint arXiv:2403.05530*, 2024. Origin of the MRCR (Multi-Round Co-reference Resolution) long-context eval.

[33] Google DeepMind. MRCR v2. `https://github.com/google-deepmind/eval_hub/tree/master/eval_hub/mrcr_v2`, 2026. Harder 2/4/8-needle variant scaling to 8M tokens; still a Google DeepMind eval [32] despite appearing in OpenAI's launch table. Accessed: 2026-07-27.

[34] Rebecca Soskin Hicks, Mikhail Trofimov, et al. HealthBench professional: Evaluating large language models on real clinician chats. *arXiv preprint arXiv:2604.27470*, 2026.

References IV

[35] Rahul K. Arora et al. HealthBench: Evaluating large language models towards improved human health. *arXiv preprint arXiv:2505.08775*, 2025.

[36] Artificial Analysis. Artificial Analysis intelligence index. https://artificialanalysis.ai/evaluations/artificial-analysis-intelligence-index, 2026. Weighted composite over 9 evals including GDPval-AA, Terminal-Bench, and HLE. Accessed: 2026-07-27.

[37] Artificial Analysis. AI coding agent benchmarks & leaderboard. https://artificialanalysis.ai/agents/coding-agents, 2026. Coding Agent Index: independently run, not vendor-reported — aggregates DeepSWE, Terminal-Bench v2, and repository Q&A, pass@1 over 3 attempts (methodology at https://artificialanalysis.ai/methodology/coding-agents-benchmarking). Scores as tabulated in [27]. Accessed: 2026-07-27.

[38] Vellum. Claude sonnet 5 benchmarks explained. https://www.vellum.ai/blog/claude-sonnet-5-benchmarks-explained, 2026. Tabulates SWE-bench Pro scores from Anthropic's Sonnet 5 System Card (2026-06-30). Accessed: 2026-07-27.

[39] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? *arXiv preprint arXiv:2310.06770*, 2024. ICLR 2024.

[40] Jatin Ganhotra. Cracking the code: How difficult are SWE-bench-verified tasks really? https://jatinganhotra.dev/blog/swe-agents/2025/04/15/swe-bench-verified-easy-medium-hard.html, 2025. Blog post, accessed: 2025-06-01.

[41] Shuvendu K. Lahiri, Sarah Fakhoury, Aaditya Naik, Georgios Sakkas, Saikat Chakraborty, Madanlal Musuvathi, Piali Choudhury, Curtis von Veh, Jeevana Priya Inala, Chenglong Wang, and Jianfeng Gao. Interactive code generation via test-driven user-intent formalization. *arXiv preprint arXiv:2208.05950*, 2022. TiCoder.

[42] Fangwen Mu, Lin Shi, Song Wang, Zhuohao Yu, Binquan Zhang, Chenxue Wang, Shichao Liu, and Qing Wang. ClarifyGPT: A framework for enhancing LLM-based code generation via requirements clarification. *Proceedings of the ACM on Software Engineering (FSE)*, 1:2332–2354, 2024. arXiv:2310.10996.

[43] Sanidhya Vijayvargiya, Xuhui Zhou, Akhila Yerukola, Maarten Sap, and Graham Neubig. Interactive agents to overcome ambiguity in software engineering. In *The Fourteenth International Conference on Learning Representations (ICLR)*, 2026. arXiv:2502.13069; later revised as "Ambig-SWE".

[44] Yuxiang Wei, Olivier Duchenne, Jade Copet, Quentin Carbonneaux, Lingming Zhang, Daniel Fried, Gabriel Synnaeve, Rishabh Singh, and Sida I. Wang. SWE-RL: Advancing LLM reasoning via reinforcement learning on open software evolution. *arXiv preprint arXiv:2502.18449*, 2025.

References V

[45] Jiayi Pan, Xingyao Wang, Graham Neubig, Navdeep Jaitly, Heng Ji, Alane Suhr, and Yizhe Zhang. Training software engineering agents and verifiers with SWE-gym. *arXiv preprint arXiv:2412.21139*, 2024.

[46] Cursor. Reward hacking is swamping model intelligence gains. https://cursor.com/blog/reward-hacking-coding-benchmarks, 2026. Accessed: 2026-06-30.

[47] AgentMarketCap. SWE-bench pro vs verified: The gap that rewrote 2026 agent procurement. https://agentmarketcap.ai/blog/2026/04/16/swe-bench-pro-vs-verified-25-point-gap-procurement-framework, 2026. Paired Verified/Pro leaderboard data, compiled April 2026. Accessed: 2026-06-30.

[48] OpenAI. Why SWE-bench verified no longer measures frontier coding capabilities. https://openai.com/index/why-we-no-longer-evaluate-swe-bench-verified/, 2026. Accessed: 2026-06-30.

[49] Cognition AI. Introducing devin, the first AI software engineer. https://cognition.ai/blog/introducing-devin, 2024. Accessed: 2026-07-09.

[50] Nam Le Hai Nam et al. On the impacts of contexts on repository-level code generation. In *Findings of the Association for Computational Linguistics: NAACL 2025*, 2025.

[51] Shukai Liu, Jian Yang, Bo Jiang, Yizhi Li, Jinyang Guo, Xianglong Liu, and Bryan Dai. Context as a tool: Context management for long-horizon SWE-agents. *arXiv preprint arXiv:2512.22087*, 2025.

[52] Pengfei Gao and Chao Peng. More with less: An empirical study of turn-control strategies for efficient coding agents. *arXiv preprint arXiv:2510.16786*, 2025.

[53] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. AutoCodeRover: Autonomous program improvement. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2024.